

PropertyLens

User Guide

How to set up and run the system on a fresh machine

READ THIS BEFORE YOU START

This guide walks a new user from a fresh git clone all the way to a running app. Total time is around 30 to 60 minutes depending on your internet speed. The bulk of that is downloading model artifacts (~1.5 GB) from Hugging Face. **If you only have 10 minutes, read the Quick start checklist on the next page first.**

What you will end up with

- A backend API running on `http://localhost:8000`.
- A web app running on `http://localhost:5173` with a Buyer view, Seller view, Shortlist, and an Ask AI chatbot.
- An optional Chrome extension that scrapes a PropertyGuru listing and pre-fills the Buyer view.
- Three demo accounts already seeded in the database so you can log in immediately.

Quick start checklist

If you know your way around Python and Node, this is the entire setup in one page. Each step links to a detailed section below.

Step	What	How
1	Install prerequisites	Python 3.11, Node.js 18, Neo4j, Ollama
2	Clone repo and create venv	git clone, python -m venv .venv, pip install
3	Get a Hugging Face token	Visit huggingface.co/settings/tokens , copy a read-scope token
4	Configure .env	Copy .env.example to .env inside SystemCode/, fill in HF_TOKEN and Neo4j credentials
5	Download artifacts	Run download_artifacts_from_hf.ipynb (~1.5 GB)
6	Seed Neo4j and SQLite	Run setup_neo4j_and_db.ipynb
7	Start Ollama	ollama serve and ollama pull gemma3
8	Install frontend deps	cd frontend then npm install
9	Start backend	uvicorn main:app --reload --port 8000
10	Start frontend	npm run dev -- --port 5173 --strictPort
11	Log in	Visit http://localhost:5173 , use credentials user / 1234

LOGIN CREDENTIALS

Username: user Password: 1234

Prerequisites

Install these once on your machine before you do anything else. None of the install steps below take more than a few minutes.

Tool	Why you need it	How to install
Python 3.11+	Required for the backend. The hybrid model bundle was pickled with scikit-learn 1.8 which needs Python 3.11 or later.	macOS: brew install python@3.11 Linux: pyenv install 3.11 Windows: python.org installer
Node.js 18+	Required for the React frontend (Vite dev server) and the Chrome extension.	macOS: brew install node Linux/Win: nvm install 18
Neo4j	Powers the chatbot, Ask AI, and property search. The backend runs without it but chat features will be disabled.	Easiest: Neo4j AuraDB Free (cloud) Local: Neo4j Desktop
Ollama	Runs the local Gemma3 LLM for the chatbot. Required unless you switch to Gemini in your .env.	macOS/Linux: brew install ollama Then: ollama pull gemma3
Hugging Face token	Pulls about 1.5 GB of model and dataset artifacts on first setup. A free read-scope token is enough.	See Step 3 below for instructions

STEP 01

Get the code, set up the Python environment

Clone the repository and move into the SystemCode folder. Every command in the rest of this guide assumes you are inside SystemCode.

```
git clone <repo-url>
```

```
cd PropertyLens/SystemCode

python3.11 -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate

pip install -r backend/requirements.txt
pip install jupyter notebook ipykernel huggingface_hub
```

WHY SYSTEMCODE/ MATTERS

This repo follows the NUS-ISS IRS-PM submission template, which puts all source code under SystemCode. The backend, the setup notebooks, and the data folder all sit inside SystemCode and resolve paths relative to it. If you run setup from a different working directory, artifacts will end up in the wrong place and the backend will not find them.

STEP 02

Tell the backend where everything lives

Inside SystemCode there is a .env.example file. Copy it to .env and fill in real values.

```
cp .env.example .env
# then edit .env in your editor
```

At a minimum you must set the following four values. The next step covers how to get the Hugging Face token.

Variable	Required	Notes
HF_TOKEN	Yes	Your Hugging Face read-scope token. See Step 3.
NEO4J_URI	Yes for chat	e.g. neo4j+s://xxxx.databases.neo4j.io for AuraDB
NEO4J_USERNAME	Yes for chat	Usually neo4j
NEO4J_PASSWORD	Yes for chat	Whatever you set when you created the database
CHAT_PROVIDER	Optional	Defaults to ollama. Set to gemini if you want to use Google's API instead.
GEMINI_API_KEY	Optional	Only required if CHAT_PROVIDER is set to gemini.
JWT_SECRET	Optional	Auth signing key. Anything works for local. Change it in production.

STEP 03

This is the only step that needs an external account

YOU WILL NEED A FREE HUGGING FACE ACCOUNT

If you do not already have one, sign up at huggingface.co. It is free and takes about a minute. The token only needs read access. We use it to download model weights and feature data hosted at huggingface.co/PropertyLens.

1. Open huggingface.co/settings/tokens in your browser. Sign in if you are not already.
2. Click **Create new token**.
3. Pick the **Read** token type. Read scope is enough for this project.
4. Give the token a name you will remember (for example `propertylens-local`) and click **Create token**.
5. Copy the token. It starts with `hf_`. Paste it into your `.env` file as the value of `HF_TOKEN`.

WHERE THE ARTIFACTS LIVE

All PropertyLens artifacts are hosted at huggingface.co/PropertyLens. The setup notebook in Step 5 pulls everything you need automatically. You do not have to download anything by hand.

KEEP YOUR TOKEN PRIVATE

Never commit your `.env` file or the token itself to git. The `.gitignore` in this repo already excludes `.env`, so you should not have to think about it, but worth knowing.

STEP 04

Easiest path is the free hosted version

The chatbot, Ask AI, and property search all need a Neo4j database. You have two options.

Option A. Neo4j AuraDB Free (recommended)

1. Go to neo4j.com/cloud/aura-free and sign up.
2. Create a new free instance. Pick any region close to you.
3. Save the generated password immediately. Copy the connection URI (looks like `neo4j+s://xxxxxxxxx.databases.neo4j.io`) into your `.env` as `NEO4J_URI`.
4. Set `NEO4J_USERNAME=neo4j` and `NEO4J_PASSWORD` to whatever AuraDB generated.

Option B. Neo4j Desktop (local)

If you would rather run Neo4j locally, install Neo4j Desktop, create a new local database, and use:

```
NEO4J_URI=bolt://localhost:7687
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=<whatever you set in Neo4j Desktop>
```

STEP 05

This is the slowest step. Around 1.5 GB.

From the SystemCode folder, run the artifact download notebook:

```
jupyter notebook download_artifacts_from_hf.ipynb
```

Open the notebook and click Run All Cells. The notebook pulls the following from Hugging Face into your local data and hf_data folders.

Folder	Size	What is in it
data/artifacts/	~660 MB	Hybrid ML model bundle, SHAP cache, CBR index, Apriori rules
data/feature_data/	~870 MB	HDB feature CSVs used at inference time
data/amenities/	~5 MB	School and amenity reference data
hf_data/05_photo_layer/	~16 MB	Photo condition CNN weights (EfficientNet B0)

VERIFICATION CHECK

The final cell of the notebook prints a list of required files and a green tick or red cross next to each one. If you see any crosses, the artifact did not download. Re-run that cell. The download is idempotent so already-downloaded files are skipped.

STEP 06

Populate the knowledge graph and create the auth database

From the SystemCode folder, run the second setup notebook:

```
jupyter notebook setup_neo4j_and_db.ipynb
```

Run all cells. The notebook does four things.

- Loads or rebuilds the property knowledge base.
- Pushes about 9,710 Property nodes, 26 Town nodes, and 17 FamousSchool nodes plus their relationships into Neo4j. The operation uses MERGE so re-running is safe.
- Creates SQLite tables in data/propertylens.db for users, history, and wishlists.
- Seeds three demo users.

Username	Password	Display name
user	1234	Bhuvesh
rekha	1234	Rekha
yunchuan	1234	Yun Chuan

FIRST RUN TAKES LONGER

If the property knowledge base parquet does not exist locally yet, this notebook rebuilds it from the feature table. That adds about 5 minutes to the first run. Subsequent runs reuse the cached parquet and finish in under a minute.

STEP 07

Required for the chatbot

Open a new terminal window and leave it running for the duration of your session.

```
ollama serve          # leave this terminal running
ollama pull gemma3    # one time, ~3 GB download
```

Verify it is reachable from another terminal:

```
curl http://127.0.0.1:11434/api/tags
# should list installed models including gemma3
```

IF YOU DO NOT WANT TO RUN OLLAMA

Set `CHAT_PROVIDER=gemini` in your `.env` and provide a `GEMINI_API_KEY`. Everything else works the same. Skip this step if you do.

STEP 08

Pull the npm packages once

From the SystemCode folder:

```
cd frontend
npm install
cd ..
```

STEP 09

FastAPI on port 8000

From the SystemCode folder, with your virtualenv activated:

```
source .venv/bin/activate
cd backend
uvicorn main:app --reload --port 8000
```

Watch the startup logs. You should see successful loads of the hybrid bundle, CBR index, SHAP cache, Apriori rules, Composite SHAP explainer, cluster profiles, and Neo4j connection.

PHOTO CNN WARNING IS FINE

If you see a warning that says `condition_model_*.pth missing`, that means the photo CNN weights are not in place yet. The backend still starts. Photo features simply will not work until you re-run Step 5.

STEP 10

Vite dev server, pinned to port 5173

Open another terminal. From the SystemCode folder:

```
cd frontend
npm run dev -- --port 5173 --strictPort
```

Vite will serve the app at <http://localhost:5173>.

USE PORT 5173 EXACTLY

The Chrome extension hard-codes port 5173. If something else is using that port, Vite would normally bump to 5174 silently. The `--strictPort` flag makes Vite fail loudly instead, so you know to free the port. The extension will not work on any other port unless you edit it.

STEP 11

First successful login

Open <http://localhost:5173> in your browser. Use one of the seeded accounts.

Username `user` Password `1234`.

Things you can try once logged in.

- **Buyer view** — enter an HDB address, get a price prediction with SHAP explanations and comparable past sales.
- **Seller view** — get an asking-price recommendation and an offer-planner band.
- **Shortlist** — save listings, compare them side by side using the Versus view.
- **Insights** → **Analytics** — composite SHAP charts, cluster profiles, market trends.
- **Insights** → **Ask AI** — natural language property search. Needs Neo4j and Ollama running.

STEP 12

Scrape PropertyGuru listings into the Buyer view

BACKEND AND FRONTEND MUST BE RUNNING FIRST

The extension talks to the backend on `http://localhost:8000` and deep-links into the frontend on `http://localhost:5173`. Steps 9 and 10 must already be running before you install the extension.

1. Open `chrome://extensions` in Chrome (paste it into the address bar).
2. Toggle Developer mode ON. The toggle is in the top-right of the page.
3. Click Load unpacked in the top-left.
4. Select the `SystemCode/extension/` folder, then click Select.
5. The extension card appears with a green ON toggle. Pin it for quick access by clicking the puzzle icon in Chrome's toolbar and pinning PropertyLens.

Try it. Visit any HDB listing on PropertyGuru. The extension auto-runs at page idle and posts the listing data to your local backend. Click the pinned icon and choose Open in Buyer to deep-link into the Buyer view with the listing details pre-filled.

Troubleshooting

Symptom	Fix
Backend exits with <code>FileNotFoundError</code> on <code>hybrid_cluster_bundle.joblib</code>	Step 5 was skipped or did not finish. Re-run <code>download_artifacts_from_hf.ipynb</code> and check the verification cell at the end.

Symptom	Fix
Neo4j not available warning at startup	Either Step 6 was skipped, or your NEO4J credentials in .env are wrong. Check the URI, username, and password match what AuraDB or Neo4j Desktop gave you.
Ask AI panel returns empty or errors	Ollama is not running. Run ollama serve in a separate terminal and verify with curl <code>http://127.0.0.1:11434/api/tags</code> .
Login is rejected	SQLite tables were not created. Re-run <code>setup_neo4j_and_db.ipynb</code> . The demo users are seeded by that notebook.
Port 8000 already in use	Another process has the port. Either kill it, or run <code>uvicorn</code> on a different port (<code>--port 8001</code>) and update <code>VITE_API_BASE_URL</code> in your .env.
Photo condition card does not work	The <code>condition_model_*.pth</code> file is missing. Re-run Step 5. The notebook pulls it into <code>hf_data/05_photo_layer/artifacts/</code> .
Extension card greyed out, manifest unreadable	You selected the wrong folder. Re-select with the file picker pointing at <code>SystemCode/extension/</code> specifically, not at <code>SystemCode/</code> or <code>PropertyLens/</code> .
Inline insight does not appear on PropertyGuru	Open Chrome DevTools → Console tab and look for fetch errors. Most common cause is the backend is not on port 8000 or CORS is blocked.
Open in Buyer opens a blank page	Frontend is not on port 5173. Re-run Step 10 with <code>--port 5173 --strictPort</code> .
Extension stops working after a git pull	Reload it. Go to <code>chrome://extensions</code> and click the refresh icon on the PropertyLens card.
Backend cannot find artifacts or .env	You started the backend from the wrong directory. Activate the venv from inside <code>SystemCode</code> , then <code>cd</code> backend before running <code>uvicorn</code> .

Quick health checks

Once everything is running, these three commands confirm the system is wired up correctly.

```
# Backend liveness
curl http://localhost:8000/health

# Confirm artifacts loaded
curl http://localhost:8000/api/analytics/global-shap | head
```

```
# Confirm SQLite seeded with demo users
curl -X POST http://localhost:8000/api/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"user","password":"1234"}'
```

IF ALL THREE RETURN SUCCESSFULLY

You are good. Open <http://localhost:5173> and start using the app.

Project layout

For reference. This is what the repo looks like after setup completes. Folders marked with an arrow are pulled by the setup notebooks rather than committed to git.

```
PropertyLens/
├─ README.md                # 7-section project README
├─ ProjectReport/           # final report PDF and user guide PDF
├─ Video/                   # system modelling and use-case demo
MP4s
├─ Miscellaneous/
│   ├── docs/               # internal architecture notes
│   └─ images/              # screenshots used in report and slides
└─ SystemCode/              # all source code lives here
    ├── HOW_TO_RUN.md       # quick reference
    ├── .env                # you create this in Step 2
    ├── .env.example        # template
    ├── backend/            # FastAPI app
    ├── frontend/           # Vite plus React
    ├── extension/          # Chrome extension
    ├── notebooks/          # training pipelines
    ├── scripts/            # standalone CLI tools
    ├── data/
    │   ├── artifacts/      # ← from HF (model bundle, XAI caches)
    │   ├── feature_data/   # ← from HF (HDB feature CSVs)
    │   ├── amenities/      # in-git fixtures
    │   └─ propertylens.db  # SQLite (created by setup)
    ├── hf_data/
    │   └─ 05_photo_layer/   # ← from HF (photo CNN weights)
    ├── download_artifacts_from_hf.ipynb # Step 5
    └─ setup_neo4j_and_db.ipynb        # Step 6
```

Re-running setup safely

All three setup notebooks are idempotent. Re-running them is safe and is sometimes necessary after a git pull if the artifacts or schema have changed.

- `download_artifacts_from_hf.ipynb` `snapshot_download` skips files that are already up to date.
- `setup_neo4j_and_db.ipynb` Neo4j writes use MERGE. Demo users are seeded only if they do not already exist.
- `upload_artifacts_to_hf.ipynb` Run only when you have trained new models or refreshed feature tables. Most users never need this.

IF SOMETHING GOES WRONG AFTER A GIT PULL

First, pull the latest `.env.example` and check whether new environment variables were added. Then re-run the two setup notebooks. Most upgrade issues resolve at that point.

Please email me on bhuvesh@u.nus.edu